

WEEK 10

受控 Agent

从「答得稳的 RAG」升级成「办得对的 Copilot」——从知识问答，到能动手的自动化操作员

01 Tool

工具契约

Schema · 幂等 · 权限 · 审计

02 Route

路由 + 降级

选对一个 · 错了能降 5 级

03 HITL

人工护栏

异步 · checkpoint · 审计

04 Trace

行为血缘

五元绑定 · 100% 可重放

05 Loop

端到端闭环

Week1-10 组装成操作员

本周划重点

前 9 周练的是「答得稳」。这一周，一旦 Agent 能调工具、改工单、退款、发邮件——风险就从「答错一句话」升级成「做错一件事」。我们要做的，是把它的一个动作，都框进可控、可审计、可回滚的边界里。

LESSON 01 · TOOL CONTRACTS

工具契约化：让 Agent 的每个动作都受 Schema / 幂等 / 权限 / 审计 约束

前 9 周做「答得稳」，这一周让 Agent 真能「办得了事」

真实场景 · 一笔被扣三次的退款

我见过一个线上事故：客服 Copilot 的退款工具没做幂等，用户网络抖了一下、前端自动重发，Agent 把一笔 800 元退款连扣了三次。财务月底对账才发现——钱赔了、客户骂上微博了、内部还得追责。从那天起我立了条死规矩：写类工具不带 `idempotency_key`，一律不准上线。Agent 调工具，不是「AI 替你写代码」，是「AI 在你的生产系统里替你按按钮」——按钮底下没护栏，迟早出大事。

咱们这节聊 4 件事：

- 「工具契约」相对普通 function，多出哪 4 个工程承诺：Schema / 幂等 / 权限 / 审计
- 用 JSON Schema 写一份能上生产的工具契约（输入 / 输出 / 错误码全框死）
- 幂等为什么是 Agent 工具的「出厂标配」，3 种实现到底怎么选
- 4 类工具风险（读 / 内部写 / 外部写 / 财务）各配什么控制策略

NOT A NORMAL FUNCTION

Agent 的工具调用 $\neq$ 普通 function——它是「契约化的受控动作」

传统软件里 *function* 靠「代码 + 单测」守边界, *Agent* 调工具, 这套全失效

决定「调不调、怎么调」的, 不是你的 *if-else*, 是一个概率模型在运行时推理:

一个「看着正常」的提问

- 可能让 Agent 调一个
- 它根本不该碰的工具
- ——比如把查询调成删除

一个「看着合理」的参数

- 可能直接越过业务边界
- `refund_amount` 多写两个零
- 代码层面完全合法

一次「成功」的执行

- 可能在合规审计时
- 完全无据可查
- ——你连谁批的都说不清

工程铁律

所以工具边界必须从「代码契约」升级成「运行时强约束」→ 每一次调用, 都受 4 个工程承诺保护: Schema 强校验、幂等、权限、审计。我 review 工具上线, 这 4 项少一个, 一票否决, 没得商量。

FUNCTION VS TOOL

普通 function vs 受控 Tool——逐维度对照

说白了：受控 Tool = function + 一整套运行时护栏

维度	普通 function	受控 Tool
谁决定调用	代码逻辑（开发者写死）	LLM 推理（运行时才决定）
输入约束	语言的类型检查	JSON Schema 强校验 + 业务规则
错误处理	抛异常 / 返回 None	结构化错误码 + retry 策略
权限	靠调用方上下文	工具内嵌 actor / role 校验
幂等性	可有可无	必须（重试 / 重发是常态）
审计	打条日志事后查	审计事件（实时 + 结构化）

看懂这张表你就抓住根了 → function 的护栏在「编译期」，Tool 的护栏必须搬到「运行时」。因为决定调不调的，是个概率模型，不是你写死的 if-else。编译期拦不住一个模型临场的误判。

FOUR COMMITMENTS

受控 Tool 的 4 个工程承诺——少一个都别上线

Schema / 幂等 / 权限 / 审计，这是工具的出厂标配，不是高级特性

Schema · 强约束

- JSON Schema 定 in/out
- enum / required / format
- LLM 解码层强制
- 杜绝「格式漂移」

Idempotent · 幂等

- idempotency_key 必填
- 同 key 多次 = 一次
- 扛 retry / 双发 / 重传
- 防重复创建 / 扣款

Permission · 权限

- actor + role 内嵌
- allowed_roles 显式声明
- 调用前先校验身份
- 越权直接拒

Audit · 审计

- 每次调用写 audit
- trace_id + args_hash 必存
- 实时 + 结构化
- 合规追责的根

上线门槛

这 4 样不是「锦上添花」→ 是工具的「四证齐全」。我带团队定的红线：上生产前这 4 项缺一项，PR 直接打回，不解释。出过一次事故你就懂，这条规矩是用真金白银换来的。

REAL CONTRACT

OmniSupport 项目 · ticket_update 工具的完整契约

一份 JSON, 把一个动作彻底框死

```
{
  "name": "ticket_update",
  "description": "Update ticket status/assignee.
  Use when user explicitly edits a ticket.
  NOT for: create / delete / bulk update.",
  "input_schema": {
    "type": "object",
    "required": ["ticket_id", "actor",
      "idempotency_key"],
    "properties": {
      "ticket_id": {"type": "string",
        "format": "uuid"},
      "status": {"enum": ["pending", "in_progress",
        "resolved", "closed"]},
      "idempotency_key": {"minLength": 16}
    },
    "additionalProperties": false
  },
  "allowed_roles": ["support_agent",
    "support_lead"],
  "idempotency": "required",
  "failure_codes": ["TICKET_NOT_FOUND",
    "INVALID_TRANSITION", "PERMISSION_DENIED",
    "DUPLICATE_REQUEST"],
  "audit_fields": ["actor", "args_hash",
    "result_code", "trace_id"]
}
```

description 带「NOT for」· 给 Agent 划边界, 避免误激活 (Week 9 学过这招)

additionalProperties:false · 严格模式, 多塞一个字段就拒

idempotency_key 是 required · 从 spec 层就堵死重复执行

failure_codes 列清 · 让 Agent 调用前就知道哪些会失败、怎么接

audit_fields 强制存 · Week 14 治理直接消费这些字段

IDEMPOTENCY

幂等性是 Agent 工具的「出厂标配」——3 种实现按动作类型选

LLM 误判 / 用户重发 / Agent 重试 / 网络抖动，都会让同一个动作发起好几次

策略	怎么做	适合什么	局限
自然幂等	同 SQL 多次结果一致 (UPSERT)	status 更新、配置修改	只对可重复操作有效
idempotency_key	客户端传 key，服务端去重	建工单、付款、发邮件	服务端要维护去重表
乐观锁 + 版本号	带 version，CAS 更新	高并发更新场景	失败要重试

划重点

2024 年起的行业共识 → 写类工具默认走 idempotency_key。哪怕下游 API 不支持，也要在 Agent 层自己加一层去重（Redis + args_hash + TTL，十几行代码的事）。重复扣款这种事，发生一次就够上头条、够你写一周复盘报告——这笔保险，便宜得很。

IDEMPOTENCY CODE

OmniSupport 项目里的幂等实现——最小，但完整

一个装饰器，让所有写类工具一行升级为幂等

```
# tools/idempotency.py
import hashlib, json

class IdempotencyStore:
    """Redis-backed dedup store, TTL 24h"""
    def check_and_set(self, key, payload):
        # 请求内容也纳入指纹，避免误命中
        h = hashlib.sha256(json.dumps(
            payload, sort_keys=True
        ).encode()).hexdigest()
        ck = f"idem:{key}:{h}"
        prev = self.r.get(ck)
        if prev:
            return json.loads(prev) # 命中，回缓存
        self.r.setex(ck, self.ttl, "in_progress")
        return None # 原子占位

def idempotent_tool(fn):
    def wrapper(payload):
        prev = store.check_and_set(
            payload["idempotency_key"], payload)
        if prev:
            return {**prev, "_idempotent_hit": True}
        result = fn(payload)
        store.commit(
            payload["idempotency_key"], result)
        return result
    return wrapper

@idempotent_tool
def ticket_update(payload): ...
```

args_hash 把「请求内容」纳入指纹 · 同 key 不同参数，视为两次不同调用

check_and_set 是原子操作 · 防高并发下两个请求同时进「未命中」

TTL 24h · 平衡重复防护期和 Redis 成本，按你的业务调

_idempotent_hit 回传 · Agent 知道这次是缓存结果，可以提示用户

@idempotent_tool 装饰器 · 写类工具一行升级为幂等，无侵入

TOOL RISK

4 类工具风险——上线前的「准入检查表」

风险级别归错档、配套机制不够，事故就是迟早的事

风险类别	示例	关键风险	控制策略
Read-only	ticket_query / kpi_query	数据泄露	权限过滤（tenant / role）
Write 内部	ticket_update / assign	状态错乱	Schema + 幂等 + 审计
Write 外部	发邮件 / 推消息	不可逆 + 影响外部	HITL（Lesson 3 细讲）
Financial	退款 / 转账 / 优惠券	直接资金损失	HITL + 双签 + 审计

把这张表当「门禁」用 → 工具上线前先归类，归错一档、配套不够，就是给自己埋雷。我的经验：财务类和外部不可逆类，不管 Agent 当时多有把握、置信度多高，一律强制 HITL，没有例外——模型的「自信」从来不能当审批用。

INDUSTRY 2026

工具契约化已是 Agent 工程的事实标准——别自己造协议

2026 年三家收敛：OpenAI / Anthropic / MCP，你只管把业务工具包进去

标准 / 框架	能力	工程含义
OpenAI Structured Outputs + Function Calling	解码层强约束，只输出符合 schema 的调用	格式漂移从源头堵死
Anthropic Claude Agent SDK (2025.9 改名)	内置 tool-use loop + HITL checkpoint + MCP 客户端	契约 + 人工护栏开箱即用
MCP (2025 入 Linux 基金会 · 1 万+ server)	工具暴露标准化，跨 IDE / Agent / Server	一份契约，到处可被发现消费
OpenAI Agents SDK (2025.3)	Agents / Tools / Handoffs / Guardrails 四原语 + tracing	路由 / 兜底 / 审查成 first-class

记死

2026 年的事实 → Agent 工具的「契约层」已经标准化，三家高度收敛。你要做的不是发明协议，是把业务工具按这套契约包好。自己造一套，既费力，又被整个生态甩在后面——这账我替你算过，不划算。

LESSON 02 · ROUTING & FALLBACK

工具路由与失败回退：在 N 个工具里选对一个、错了能降级

从「单工具」到「多工具系统」——这是 Demo 到上线之间最深的那条沟

真实场景 · 把「查工单」调成了「建工单」

我接过一个项目，团队把 12 个工具一股脑全列给 LLM，prompt 就写一句「请合理选用」，demo 跑得那叫一个漂亮。上线两周，用户问「查一下我的工单」，Agent 转手去调了「创建工单」，凭空生成一堆空单，客服一早上都在清垃圾数据。这不怪模型笨——是你把一个高确定性的路由问题，扔给一个概率模型去赌。我跟你讲，工具一过 5 个，自由选就开始误选，这几乎是个规律。

咱们这节聊 4 件事：

- LLM 自由选 / 规则+LLM / 显式 Workflow，三种调度策略的边界在哪
- 设计「主链路 + N 级降级」的 fallback chain，失败也优雅
- 把 Week 8 的 RAG 服务也包装成工具，让 Agent 统一编排
- 跑通「知识问答 + 指标查询 + 工单更新」三工具协同

FRAMEWORK OVER FREEDOM

生产级 Agent 不让 LLM 「自由发挥」——给它「明确的决策框架」

Demo 能跑通常见 case, 一上线就开始翻车

把所有工具列给 LLM、prompt 写「请合理选用」，上线后你会撞上这三件事：

工具数到 5+

- 误选率开始飙
- 查询调成写入
- 看着合理，全是坑

工具调失败

- LLM 经常陷入
- 无限重试
- 会话资源被占死

多步编排

- 复杂任务里
- LLM 偶尔「漏掉一步」
- 结果你还查不出来

工程铁律

问题不是 LLM 不够聪明 → 是「让 LLM 决定所有事」，等于主动放弃了工程师最大的武器：把「高确定性」的部分用代码写死，只把「真需要判断」的那个窄决策，留给模型。该写死的别交给模型，这是受控 Agent 的第一性原理。

THREE STRATEGIES

工具调度的 3 种策略——按确定性高低选

两端都是极端，默认从中间那档起步

LLM 自由选 · FREEFORM

- 全部工具暴露给 LLM
- LLM 推理选哪个
- 适合：工具<5 + 场景多变
- 局限：误选率高、不可控
- 工程负担：最低

规则 + LLM · HYBRID

- 关键词/意图先预过滤
- 过滤后再让 LLM 决策
- 适合：工具 5-20
- 局限：规则要维护
- 工程负担：中（推荐默认）

显式 Workflow

- 代码定义流程节点
- LLM 只在节点内窄决策
- 适合：高价值+流程清晰
- 局限：灵活性差
- 工程负担：最高

关键判断

默认从「规则 + LLM 兜底」起步 → 左右兼顾。全自由 = 不可控，全代码 = 没弹性，这两端都是新手才会选的极端。



WHEN TO USE WHAT

3 种策略的选择标准——看「确定性 × 风险」

风险越高、确定性越高，越往代码侧靠

场景	确定性	风险	建议策略
客服 FAQ 问答	低	低（答错）	LLM 自由选
多产品线咨询路由	中	中（误导用户）	规则 + LLM 兜底
工单状态更新	高	中（数据错乱）	规则 + LLM（带 confirm）
退款 / 资金动作	高	高（资金损失）	显式 Workflow + HITL
多步骤报表生成	中	低	显式 Workflow

选型口诀 → 风险低就让 LLM 放手跑，风险一高就把方向盘抢回代码手里。退款这种事，宁可流程僵一点、体验差一点，也绝不能让模型「自由发挥」——僵一点最多挨句抱怨，放飞一次就是真金白银。

FALLBACK CHAIN

工具会失败——生产级 Agent 要有 5 级降级链

不是「上抛异常 → LLM 临场道歉」，是每一类失败都显式声明降级路径



关键判断

5 级 fallback 不是过度设计 → 是生产 Agent 的 SLO 地基。每一级都对应一个具体工程对象（独立函数 + 独立超时 + 独立 audit），不是「LLM 临场决定降到哪」。降到第几级，日志里一目了然——事后你能精确说出「那天它走到了 cache 层，因为主 API 超时了 1.5 秒」。

FALLBACK CODE

OmniSupport 项目 · ticket_query 的 fallback 实现

每一层独立、可定位、自带 audit

```
@register_tool(name="ticket_query")
def ticket_query(payload):
    chain = FallbackChain([
        # 1. 主链路: 实时 API
        ("primary", primary_api,
         {"timeout_ms": 1500}),
        # 2. 重试: 同链路 3 次, 指数退避
        ("retry", primary_api,
         {"retries": 3, "backoff": "exp"}),
        # 3. 备用: 5 分钟前缓存 (标记 stale)
        ("cache", cached_lookup,
         {"max_age_seconds": 300,
          "mark_stale": True}),
        # 4. HITL: 异步推 Slack, 不阻塞 Agent
        ("hitl", slack_notify_oncall,
         {"channel": "support-oncall",
          "priority": "P2"}),
        # 5. Graceful: 返回降级响应
        ("graceful", lambda p: {
            "success": False,
            "user_message": "系统繁忙, 已通知后台",
            "fallback_level": "graceful"}, {}),
    ])
    return chain.execute(payload)
# 每一级都写 audit:
# {"fallback_level": "cache",
#  "primary_error": "TIMEOUT"}
```

每级独立函数 + 独立超时 + 独立 audit · 精确定位降到了哪一级

cache 层 mark_stale=True · Agent 知道是缓存, 可以告知用户

HITL 异步推 Slack 不阻塞 · 人慢慢处理, Agent 继续走 graceful

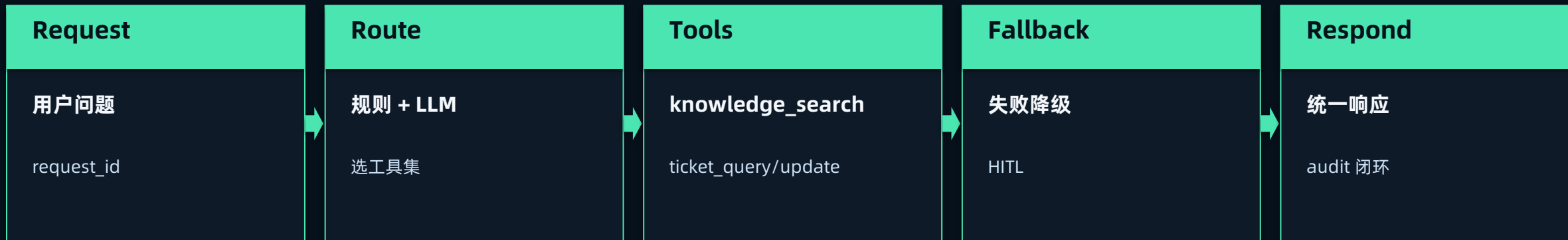
graceful 返回 user_message · 前端直接显示, 不让 Agent 临场编道歉

所有降级动作进 audit · Week 14 治理直接消费

RAG AS A TOOL

RAG 也是一种工具——让 Agent 用同一套路调度

很多团队 RAG 一套、Tool 一套，这是 Week 8/10 之间最常见的工程债



关键判断

把 RAG 也包成工具 → 同一套 schema、同一套 fallback、同一套 audit。当 RAG 也只是「一个工具」，Agent 的世界就只剩一种调用——这是 Week 10 系统性思考的核心：抽象统一了，治理才能统一。两套系统并行，迟早两套都治不好。

RAG-AS-TOOL CONTRACT

把 Week 8 的 RAG 服务「工具化」的契约

从这一刻起，RAG 是 Agent 的一个普通工具，不是特殊系统

```
{
  "name": "knowledge_search",
  "description": "Search internal KB.
  Returns ranked chunks with evidence
  anchors. Use for 'how to / what is'.
  NOT for: real-time data, ticket info,
  financial queries.",
  "input_schema": {
    "required": ["query", "tenant"],
    "properties": {
      "query": {"type": "string",
        "maxLength": 500},
      "top_k": {"type": "integer",
        "minimum": 1, "maximum": 10},
      "filters": {"doc_type":
        {"enum": ["faq", "manual", "policy"]}}
    }
  },
  "output_schema": {
    "evidences": {"type": "array",
      "minItems": 1,
      "items": {"$ref": "#/EvidenceAnchor"}},
    "confidence": {"type": "number"}
  },
  "allowed_roles": ["any_authenticated"],
  "idempotency": "natural",
  "fallback": ["primary->retry->cache->graceful"]
}
```

description 带「NOT for」· Agent 不会在查工单时误调 KB 搜索

evidences.minItems=1 · 和 Week 8 契约一致，强制证据引用

idempotency: natural · 查询天然幂等，不需要 key

fallback 复用 Lesson 2 标准模式 · 降级链一套到底

这一刻起，RAG 架构极简：Agent 眼里只剩一种调用

INDUSTRY 2026

Agent 编排框架收敛——纯「自由 Agent」生产里几乎不存在

多工具协同已是标配，Workflow + 受控工具才是上线形态

框架 / 实践	能力	工程含义
OpenAI Agents SDK (2025.3)	Guardrails 与执行并行跑、fail fast + handoffs + tracing 内置	loop 超限自动跳出 + 升级人工
LangGraph	有状态图 + interrupt 中断节点 + durable checkpointer	Agent 在节点上「暂停→等→恢复」
Anthropic: Workflow vs Agent	官方明确区分确定性 Workflow 与自主 Agent	生产默认 Workflow，关键场景才上 Agent
MCP (1 万+ server)	工具服务化跨平台标准	Agent 可发现并消费任何 MCP server

关键判断

2026 年的事实 → 纯「自由 Agent」在生产里几乎不存在。OpenAI Agents SDK 把路由 / 兜底 / 人工审查做成 first-class，正是承认了这件事：能上线的版本 = Workflow + 受控工具 + HITL。别被 demo 视频骗了。



LESSON 03 · HUMAN IN THE LOOP

HITL：给 Agent 装「不可逾越的人工护栏」——但不能拖死系统

退款 / 删数据 / 发对外邮件 / 改财务，再稳的 Agent 也不能自己拍板

真实场景 · 一个把系统卡死 40 分钟的 HITL

我见过最离谱的 HITL：团队给高额退款加了个「确认弹窗」，Agent 同步等审批。结果审批人正在开会，Agent 就那么挂了 40 分钟，占着会话资源不放，后面排队的用户全超时。HITL 做错了，比不做还糟——它会反过来卡死你整个系统。真正的 HITL 不是「加个按钮让人点」，是工程化的「决策权移交」：可观测、可追责、可回放，而且 Agent 绝不能在等审批时挂死。

咱们这节聊 4 件事：

■ 哪些动作必须 HITL：3 类强制 + 2 类条件触发

■ 异步 HITL workflow：Agent 不卡死、审批方不被打扰

■ 4 种 HITL 模式（pre-confirm / approval / review / escalation）

■ HITL 决策接进 Week 14 的 release manifest 审计闭环

HANDOVER, NOT A BUTTON

HITL 不是「加个按钮让人点」——是工程化的「决策权移交」

UI 上的 HITL，和工程化 HITL，差着好几层

只做个弹窗，你会缺掉这四样，每一样都能让你上线后翻车：

没有 SLA

- 审批人
- 没看到怎么办？
- 没人定超时

没有降级

- 超时了
- 走哪条路？
- 没预案就挂死

没有审计

- 谁批的、
- 何时批、为什么批？
- 合规来查全抓瞎

没有异步

- Agent 等审批时
- 占着资源不放
- 并发直接打满

关键判断

真正的 HITL = 决策权移交协议 → 把「AI 自动 → 人决定」这条边界，做成可观测、可追责、可回放的工程对象。弹窗只是它的皮，里子是 SLA + 降级 + 审计 + 异步这四样。皮谁都会画，里子才是分水岭。

UI VS ENGINEERED

UI 上的 HITL vs 工程化 HITL——逐维度对照

Demo 阶段够用，生产环境必须升级

维度	UI HITL（常见但不够）	工程化 HITL（生产级）
形态	弹窗 / 确认按钮	审批 workflow + audit event
同步性	阻塞 Agent 等待	异步 + Agent 释放资源
SLA	没有这个概念	明确 timeout + 降级
审计	没有，或事后补	审批前 / 中 / 后全程 audit
决策回放	不可能	完整可重放（who / when / why）
Agent 状态	挂在那干等	存 checkpoint，可恢复

直说了 → UI HITL 在 demo 里很好看，一上线就两个死法：要么 Agent 挂死，要么被悄悄绕过审批。区别全在「异步 + checkpoint + audit + SLA」这四样有没有。没有，它就是个好看的摆设。

THREE STAGES

HITL 三阶段 workflow——决策权移交全过程

Pre → Wait → Post, 每一段都有 audit

Pre-Decision

- Agent 准备调工具
- 检测到风险触发
- 生成 approval_request
- 附完整上下文
- 冻结 checkpoint

Wait & Notify

- 推送审批人 (Slack/邮件)
- 设 SLA 超时 (如 15 分钟)
- Agent 不阻塞, 切别的会话
- 审批人可拒绝 + 写理由
- 审计 notified_at

Post-Decision

- 审批结果触发 callback
- Agent 从 checkpoint 恢复
- 执行 / 拒绝 / 升级
- 全程写 audit_event
- 回放: 谁批、为什么

这套机制有个工程名字 → Checkpoint + Resume。冻结现场、释放资源、等回调、恢复继续。LangGraph 的 interrupt 干的就是这件事, 别自己从零造。



WHEN TO TRIGGER

HITL 触发条件——3 类强制 + 2 类条件

必须代码强制校验，绝不能依赖 LLM 自己判断

类别	典型动作	触发原因	审批层级
强制 · 财务	退款 / 转账 / 优惠券	资金不可逆	财务团队
强制 · 不可逆	删数据 / 删文件 / 发对外邮件	执行后无法回滚	业务负责人
强制 · 合规	导出 PII / 跨域查询 / 法务报告	合规风险	合规团队
条件 · 金额阈值	退款 > 1000 元	超自动审批额度	主管
条件 · 异常模式	Agent 一小时 5 次重试	可能被滥用 / 攻击	安全团队

关键判断

这张「触发矩阵」必须团队明列、Agent 框架强制校验 → 千万别写成「让 LLM 判断要不要审批」。模型在某些刁钻措辞上会漏审，而漏审一次，可能就是一笔不可逆的资金损失。规则的事，交给规则。



ASYNC HITL CODE

OmniSupport 项目里的异步 HITL 实现

Checkpoint + Resume: Agent 不被卡死的关键

```
def request_approval(req, state):
    # 1. 审计: 审批发起
    audit("hitl_requested", req)
    # 2. 存 checkpoint, 立即释放资源
    checkpoint_store.save(
        key=req.request_id, agent_state=state,
        pending_action=req,
        expires_at=now()+req.sla_minutes*60)
    # 3. 异步推审批人, 不阻塞
    notify_approver(layer=req.approval_layer,
        callback_url=
            f"/api/hitl/{req.request_id}/decide")
    # 4. 控制流还给 Agent: 去处理别的会话
    return {"status": "awaiting_approval",
        "user_message": "已转主管审批, 约 15 分钟回复"}
```

```
@app.post("/api/hitl/{rid}/decide")
def hitl_decide(rid, decision, approver, reason):
    check_role(approver, req.approval_layer)
    audit("hitl_decided", rid,
        approver, decision, reason)
    state = checkpoint_store.load(rid) # Resume
    agent.resume(state, decision=decision,
        approver=approver, reason=reason)
```

checkpoint_store.save 序列化 Agent 状态 · 暂停+存现场, 资源立放

notify_approver 异步推 · Slack / 邮件 / 移动端, 方便时再看

callback_url 让审批人走 API 决策 · 不在 Agent 进程里阻塞

audit 写在发起 / 决策 / 执行三时点 · 完整回放链

agent.resume 恢复继续 · 用户几乎感受不到中断

FOUR PATTERNS

4 种 HITL 模式——按「何时介入」分类

不是替代关系，是「动作风险 × 检查时点」的笛卡尔积

模式	介入时点	典型场景	工程实现
Pre-Confirm	工具执行前	高额退款、删数据	request_approval + 阻塞
Approval	执行后、生效前	邮件发送（草稿审）	生效前 hold + 异步审
Review	执行后采样审	内部数据导出（10% 采样）	执行 + 异步采样人工审
Escalation	Agent 不确定时	confidence < 0.7	LLM 主动触发

挑模式就看两件事 → 动作可不可逆、检查放执行前还是后。不可逆的（退款 / 删数据）必须 Pre-Confirm 卡在前面；可逆但要质量的（数据导出）用 Review 采样审，别让审批拖慢主流程。Escalation 是给「模型自己心虚」时留的逃生口——它说不准，就别硬撑。

DESIGN PRINCIPLES

HITL 设计的 5 条核心原则——全是真实生产事故反推出来的

1 · 审批 1 屏内能决策

给摘要 + 关键参数 + 一键批/拒 + 拒绝必填理由 · 决策时间压到 30 秒内

2 · 拒绝必须填理由

「为什么拒」是免费的训练数据，回流优化 · 攒半年能把误触发率砍一半

3 · 审批不逐级走

「组长→总监→VP」是地狱设计 · 改成「分层级 + 单层决策」

4 · 超时必有降级

SLA 15 分钟没批：默认升级 / 默认拒绝 / 转人工 · 提前定义死

5 · 审批 audit 不可篡改

写 append-only 存储 · 谁批、何时、何理由 · 合规审查的根

关键判断

原则 2 最被低估 → 拒绝理由是免费的训练数据，攒半年能把误触发率砍一半。别让审批人「点一下就走」。

ANTI-PATTERNS

HITL 设计的 5 个常见反模式

最隐蔽、也最致命的，是「让 LLM 判断要不要审批」

反模式	后果	正确做法
同步等审批	资源浪费 + 用户干等	异步 + checkpoint
没有超时	事务挂死	SLA + 降级路径
审批信息太长	审批慢 / 误批	摘要 + 关键参数
不写审批 audit	合规审查直接失败	发起/通知/决策三段 audit
让 LLM 判断是否 HITL	不一致 / 漏审	规则强制 + 触发矩阵

踩坑提醒

反模式 5 最坑·团队以为「AI 自己会决定要不要审批」，结果某些 case 完全漏审——我见过一个 Agent 因为用户 query 措辞特殊，绕过了财务审批直接退款，事后复盘才发现这个口子。HITL 触发必须是代码强制，不是模型自觉。模型的「自觉」，撑不起合规。

INDUSTRY 2026

HITL 已从「建议做」升级为「必须做」

主流框架都把人工介入做成了 *first-class*

框架 / 标准	能力	工程含义
LangGraph interrupt	每个 super-step 存 StateSnapshot, 需 durable checkpointer	checkpoint≠durable, 生产要 Postgres/DynamoDB + 失败恢复
OpenAI Agents SDK	human approval 内置, 控制平面在 trusted infra	auth / audit / 人审在容器外, 可恢复
Anthropic HITL 三模式	approve before / sample after / escalate when uncertain	官方推荐为 Agent 设计核心
Claude Computer Use	删文件 / 提交表单等高危, 强制人工确认	HITL 是默认行为, 不是可选项

关键判断

2026 年的事实 → 企业级 Agent, HITL 是准入条件, 不是加分项。提醒一句: LangGraph 那个 checkpoint 只是状态持久化, 业界 2026 已经在喊「checkpoint ≠ durable execution」——生产还得自己补失败检测和恢复逻辑。

LESSON 04 · ACTION LINEAGE

Agent 行为血缘：从「做了什么」追到「为什么这么做」+「数据从哪来」

Week 6 做完数据血缘，Agent 时代要扩展到「动作血缘」

真实场景 · 一笔复现不出来的错误退款

有次客户投诉：「上周三你们 Agent 给我算的退款额不对。」团队查了一整天日志也复现不出来——因为当时的数据快照、Prompt 版本、模型版本，全没记。最后只能赔钱了事，还赔得不明不白。那天我算彻底想明白了：Agent 没有「可重放的现场」，出了事，你连「它当时为什么这么做」都答不上来。行为血缘要解决的，就是把一个动作和它依赖的全部上下文，绑成一个能 100% 重放的现场。

咱们这节聊 4 件事：

- 数据血缘 / 调用链 / 行为血缘，三种 trace 的覆盖范围和差异

- Agent 动作的「五元绑定」：数据快照 + 证据 + Prompt + 工具 + 模型

- Agent 行为 trace 接进 Week 6 OpenLineage + Week 14 release manifest

- Action Replay：点开一个 Agent 动作，一屏看清全部上下游

DECISION AS ASSET

Agent 行为血缘 = 把「AI 的每次决策」当顶层资产纳入血缘图

传统血缘是「表→表」，Agent 血缘是「动作→6 个维度的依据」

Agent 改了某个工单，你要能立刻回答它到底依据了什么——

问题与证据

- 哪份用户问题
- 哪份索引 / 哪份切片
- ——答案的源头

Prompt 与工具

- 哪个 Prompt 版本
- 哪个工具 / Skill 版本
- ——行为的约束

模型版本

- 哪个模型版本
- claude-sonnet-4-6
- ——推理的大脑

关键判断

这是 6 个维度的血缘，不是一条单线索 → Agent 时代的血缘必须「扩展」：把数据血缘 + 行为血缘 + 模型/Prompt 版本，全部统一在同一张图里。少绑一维，复盘就多一个「说不清」，监管就多一个抓你的把柄。

THREE TRACE LAYERS

三层 Trace——各自的覆盖范围与组合

同一个 `trace_id`, 把三层串成一条线

Data Lineage · 数据

- Week 6 讲过
- 表 / 文件 → 上游
- 工具: OpenLineage
- 答: 这表数据怎么来的
- 粒度: 表级 / 字段级

Call Trace · 调用

- Week 12 专题
- 请求 → 服务 → 服务
- 工具: OpenTelemetry
- 答: 这次请求花在哪段
- 粒度: 服务级 / span 级

Action Lineage · 动作

- 本课重点
- 动作 → 决策依据 全链路
- 组合: OL + OTel + audit
- 答: 为什么 AI 这么做
- 粒度: 动作级 + 依据

别重造轮子 → Action Lineage 不是新建一套系统, 是用同一个 `trace_id` 把已有的 OpenLineage (血缘) + OpenTelemetry (span) + audit (合规) 三层缝起来。三样你早就有了, 缺的只是那根串起来的线。

WHAT TO BIND

Agent 动作的「五元绑定」——一个动作 = 5 类上下文

5 类一起绑定 = 完整的「动作可重现性」

维度	绑定什么	为什么	从哪来
Data Snapshot	iceberg_snapshot_id	数据快照锁定	Week 4 Iceberg V3 time travel
Evidence Chunks	chunk_id + doc_version	具体引用证据	Week 7 evidence_anchor
Prompt Version	prompt_release_id	LLM 行为约束	Week 8 Prompt as Code
Tool/Skill Version	tool_v + skill_pack_release	动作执行约束	Week 9 SKILL.md
Model Version	claude-sonnet-4-6	推理模型版本	Week 8 release manifest

关键判断

五元绑定的价值就一条 → 出问题能 100% 复现当时的现场。少绑一个维度，复盘就多一个「说不清」。我的标准：这 5 样绑不齐，这个动作就不算「可追责」，也就不该上生产。可追责，是受控 Agent 的底线。

TRACE EVENT

OmniSupport 项目 · Agent 动作的 trace event 真实结构

一个 event, 把「做了什么 + 为什么」全装下

```
{
  "event_type": "agent.action.completed",
  "trace_id": "trace-20260518-001",
  "session_id": "sess-user-42-conv-87",
  "action": {          # 做了什么
    "type": "tool_call",
    "tool_name": "ticket_update",
    "tool_version": "1.4.0",
    "args_hash": "sha256:abc123...",
    "result_code": "SUCCESS"
  },
  "reasoning": {       # 为什么这么做
    "user_query": "把工单 T-1234 改 in_progress",
    "evidences": [{"chunk_id": "chk-faq-042",
      "doc_version": "v3.7.2"}],
    "data_snapshots": {"iceberg.tickets":
      "snap_2026_05_18_093200"}
  },
  "bindings": {        # 五元绑定
    "release_id": "rag-v2026.05.18-001",
    "prompt_release_id": "v3.2.1",
    "model_version": "claude-sonnet-4-6",
    "skill_pack_release": "1.4.0"
  },
  "approval": {"approver": "alice@support"}
}
```

action 描述「做了什么」· 和 Lesson 1 工具契约的 audit_fields 对应

reasoning 描述「为什么」· query + evidences + snapshots, 回放当时依据

bindings 是五元绑定 · release/prompt/model/skill/tool, 出事秒查现场

approval 串接 Lesson 3 的 HITL audit · 形成完整责任链

这条 event 进 OpenLineage / DataHub · 每个动作一屏看清

UNIFIED PIPELINE

动作血缘怎么「接进」现有治理栈——不重造轮子

三类工具，发同一个 `trace_id`



关键判断

别为 Agent 重建一套血缘 → 把它接进 Week 6/8/9 已经建好的治理栈。OpenLineage 发血缘、OpenTelemetry 发 span、audit 发审计，三者用同一个 `trace_id` 串起来——这才叫「统一可观测」。重建一套新系统的人，最后维护的是两套都不完整的系统。

OPENLINEAGE CODE

把 Agent 动作发到 OpenLineage——复用 Week 6 基础设施

Agent 动作和数据资产，连成同一张 graph

```
# agent/lineage.py
from openlineage.client import (
    OpenLineageClient, RunEvent)
from openlineage.client.run import (
    RunState, Dataset, Job, Run)

def emit_agent_action(event):
    ol.emit(RunEvent(
        eventType=RunState.COMPLETE,
        run=Run(runId=event["trace_id"],
            facets={"parent":
                ParentRunFacet.create(
                    runId=event["session_id"],
                    name="agent_session"))),
        job=Job(namespace="omnisupport.agent",
            name=f"action."
                f"{event['action']}['tool_name']}"),
        # 输入：决策依赖的数据 + 切片
        inputs=[Dataset(namespace="iceberg", name=s)
            for s in event["reasoning"]
                ["data_snapshots"].values())
            + [Dataset(namespace="rag.chunks",
                name=e["chunk_id"])
                for e in event["reasoning"]["evidences"]],
        # 输出：动作改变的资产
        outputs=[Dataset(namespace="iceberg.tickets",
            name=event["action"]["target_id"]),
        ])
    )
```

OpenLineage 是 Week 6 已在用的标准 · 不用重造，直接复用

inputs 列上决策依赖的数据 + 切片 · 下游知道这次动作的来源

outputs 列上动作改变的资产 · 反向血缘：哪些动作改过这张表

parent_run = session_id · 一次会话多个动作，组成血缘子图

DataHub / Marquez / OpenMetadata 上 · Agent 动作和数据资产连成统一 graph

ACTION REPLAY

血缘最大的价值不是「查」——是「重放」

客户投诉「上周三退款额错了」，你能 5 分钟重放当时现场吗

场景	需要什么	没血缘的代价
客户投诉复盘	重放当时现场 + 决策依据	人工查日志几小时 / 几天
Bug 回归	改完代码跑历史 case 看是否好转	没法对比，全靠猜
合规审计	为什么这条决策符合监管	审查直接失败
模型对比	Claude Sonnet 4.6 vs Claude Opus 4.8 严格 A/B	不能做严格对比

踩坑提醒

Action Replay 是企业级 Agent 上线的「事故复盘必备能力」· 没有它，Agent 就是个黑盒，迟早被监管打回——我赔过那笔复现不出来的退款，从那以后，「五元绑定 + 可重放」是我给 Agent 定的硬门槛，不是锦上添花。黑盒能跑 demo，但扛不住一次真投诉。

INDUSTRY 2026

Agent 行为血缘是 2026 治理与合规的硬要求

可解释 / 可审计 / 可重现, 工程基础都是 Action Lineage

标准 / 法规	要求	工程含义
EU AI Act · 自动日志	高风险 AI 全生命周期自动记录日志, 留存 ≥ 6 个月	Art.19/26 · 没有行为血缘几乎无法满足
EU AI Act · 新时间表	2026.5 Digital Omnibus → 高风险 Annex III 推迟到 2027.12.02	罚则 €15M 或全球营收 3% (取高)
OpenTelemetry GenAI 语义约定	gen_ai.* 统一 invoke_agent / execute_tool span	2026 仍在 experimental→stable 过渡
Phoenix / Langfuse / LangSmith / TruLens	OTel-native 一站式 trace + reasoning + tool	动作 + 决策依据可查可比

关键判断

2026 治理共识 → 「可解释 / 可审计 / 可重现」三件事的工程基础, 都是 Action Lineage。Anthropic Citations API 已把「决策依据」做成产品级能力。EU AI Act 虽把高风险大限延到 2027 底, 但「自动日志 + 留存 6 个月」这条没松——提前把血缘建好, 到时候不慌。

LESSON 05 · END-TO-END LOOP

端到端 Copilot 闭环：把 Week 1-10 的所有能力，组装成一个能办事的 Agent

从「答得稳」到「办得对」的最后一英里

真实场景 · demo 惊艳、一上线就崩

我跟你讲个扎心的事实：让 Copilot 跑通一个 demo，三天；让它真上线办业务，前面那 9 周一周都不能少。我见过太多团队 demo 惊艳、一上线就崩——因为他们跳过了契约、跳过了血缘、跳过了 HITL。没有 Week 2 契约，数据不可信；没有 Week 8 RAG 服务化，答案没证据；没有 Week 10 的工具契约+HITL+血缘，动作没边界。系统工程没有捷径，少任何一周，Copilot 都上不了真实业务。

咱们这节聊 4 件事：

- 端到端架构图：Week 1-10 成果的统一视图
- Copilot 演示必须覆盖 3 类路径：成功 / 失败 / HITL
- 为 Copilot 上线建立 5 个核心 SLO 指标
- Copilot 上线后 12 个月的演进路线（Week 11+ 衔接）

LAUNCH ≠ DEMO

Copilot 上线 ≠ Demo 跑通——是把 10 周工程承诺串成一条业务可用的操作链

少了哪一周	会缺什么	后果
Week 2 契约	Agent 拿到的数据不可信	答案地基是流沙
Week 6/7 资产化 + 多模态	数据不稳定、答不了复杂问题	一问深就露馅
Week 8 RAG 服务化	Agent 答案没有证据	合规第一关就过不了
Week 9 Skill 标准	Agent 没有可治理的工艺	想优化都无从下手
Week 10 契约+HITL+血缘	Agent 的动作没有边界	一动手就是事故

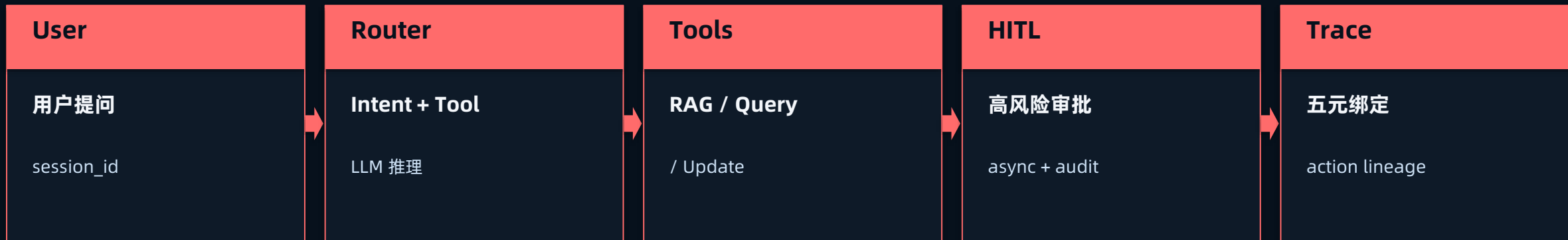
关键判断

Demo 三天能跑通，上线要前面整整 10 周 → 这就是「系统工程」的分量：能办事、办错可追责、上游变更不崩、合规审查能过。这四样，没有一样是 demo 那三天能攒出来的。

COPILOT ARCHITECTURE

OmniSupport Copilot 端到端架构——10 周成果的统一视图

Agent 不是黑盒，是一串工程对象的有序组合



关键判断

这张图把 Week 1-10 的工程对象汇成一条业务链 → Agent 这才真正成为企业 workflow 里的「自动化操作员」，而不是一个聊天机器人。每一段都可观测、可审计、可降级。你能指着任意一段说清「它在干嘛、出事怎么办」——这就是「受控」两个字的全部含义。

DEMO PATHS

Copilot 端到端演示——3 类必跑 + 2 类进阶路径

只跑 happy path 是 demo 级，跑全才是生产级

路径	场景	验证什么	关键检查点
Happy	问「如何重置密码」	RAG → 答案 + 五元绑定	evidences ≥ 1，引用准确
Fallback	工单 API 超时	fallback → cache → graceful	fallback_level audit
HITL	退款 5000 元	Pre-Confirm → 异步审 → 执行	approval audit + checkpoint
Composite	查 KB + 改工单	workflow 编排 + trace 关联	单 trace_id 关联多动作
Replay	上周投诉案例	Action Replay 重放现场	复现当时决策

这 5 类路径每个都要写成自动化测试 + 演示脚本 → 上线前 100% 通过，上线后定期回归。我的硬规矩：fallback 和 HITL 这两条路径没跑绿，绝不让 Copilot 碰真实业务——因为出事的，永远不是 happy path。

E2E DEMO SCRIPT

OmniSupport Copilot 端到端演示脚本（HITL 路径）

assert 不是「看看就好」，是发布前必跑的自动化测试

```
# demos/e2e_hitl_path.py
async def demo_hitl_refund_path():
    """客户要 5000 元退款 → 触发 HITL"""
    copilot = OmniSupportCopilot(
        release_id="rag-v2026.05.18-001") # W8/9
    # Step 1: 用户提问
    resp = await copilot.ask(session_id="demo-001",
        query="帮我处理订单 ORD-789 退款，5000 元")
    # 检查 1: 应触发 HITL，不应直接执行
    assert resp["status"] == "awaiting_approval"
    assert resp["approval_request"]["risk_category"] \
        == "financial"
    rid = resp["approval_request"]["request_id"]
    # Step 2: 模拟审批人决策
    await copilot.approve(request_id=rid,
        approver="finance-lead", decision="approve",
        reason="VIP 客户、订单合理、已核对发票")
    # 检查 2: Agent 应自动恢复并执行
    final = await copilot.poll_completion(rid)
    assert final["action"]["result_code"] == "SUCCESS"
    # 检查 3: 五元绑定 + audit trail
    assert len(final["audit_chain"]) >= 3
    print("OK · HITL Path 闭环验证通过")
```

一次 demo 验了 7 个工程对象 ·
release/risk/audit/approver/bindings/trace

assert 是自动化测试 · 每次发布前必跑，不绿不上线

poll_completion 是异步 HITL 的客户端等待 · 不阻塞 Agent

audit_chain ≥ 3 · 发起 / 审批 / 执行三段都要 audit

这就是前面 9 周所有工程承诺的一次「压测」



COPILOT SLO

Copilot 上线的 5 个核心 SLO——上线 ≠ 跑通

少一个指标，团队就会在「看不见的故事」里翻车

指标	含义	目标值	违反的代价
Resolution Rate	一次解决率	> 65%	工单成本 / NPS 下降
Evidence Coverage	答案带证据率	= 100%	合规失败 / 信任流失
Tool Success Rate	工具调用成功率	> 98%	用户体验断崖
HITL SLA	审批 15 分钟内响应率	> 95%	事务挂死
Hallucination Rate	幻觉率（抽样）	< 2%	风险 + 合规

关键判断

这 5 个指标必须有专属 Grafana 面板 + 阈值告警 → 它们是 Copilot 是否「真正上线」的硬约束。Evidence Coverage 必须卡 100%，因为差一条没证据的答案，可能就是一次合规事故——这个指标，没有「大概」。

INDUSTRY 2026

Copilot 上线 = 受控 + 可解释 + 合规——这是准入条件，不是高级技巧

监管在驱动 Copilot 的工程化标准

框架 / 法规	要求 / 能力	工程含义
OpenAI Agents SDK (2025.3)	工具路由 / fallback / tracing first-class	生产部署事实标准
LangGraph for Production	stateful workflow + interrupt + durable	生产 Agent 事实编排框架
Anthropic: Workflow over Agent	生产用 Workflow + 受控工具 + HITL	全自由 Agent 几乎不存在
EU AI Act + 中国生成式 AI 备案	强制审计 / HITL / 行为追溯	监管在倒逼 Copilot 工程化

关键判断

2026 年的事实 → Copilot 上线必须满足「受控 + 可解释 + 合规审计」。这些不是「高级技巧」，是「准入条件」。EU AI Act 罚到营收 3%、中国要算法备案——工程化不是选择题，是入场券。

ROADMAP AHEAD

Copilot 上线后 12 个月的演进路线——Week 11-15 衔接

Week 10 是里程碑，但工程治理才刚刚开始

阶段	核心动作	对应周次
1-3 月 · 稳定	SLO 监控 / 评测体系建立	Week 11 评测、Week 12 可观测
3-6 月 · 扩展	GraphRAG 跨文档归纳	Week 13 GraphRAG
6-9 月 · 治理	统一版本 / 灰度 / 回滚	Week 14 治理
9-12 月 · 商业化	成本 / 性能 / SLO 全打通	Week 15 Capstone

Week 10 是「Copilot 上线」的里程碑 → 但上线只是开始，真正的难度在持续运营。先把 SLO 监控和评测（Week 11）搭起来，否则你连「它今天有没有变差」都不知道。我见过太多团队上线完就撒手，三个月后质量悄悄滑坡，等用户投诉了才发现——别犯这个错。

WEEK 10 MAP

Week 10 五节的完整能力栈——一个「能办事的 Copilot」

从工具契约，到端到端闭环



关键判断

这 5 节构成「受控 Agent」的完整能力栈 → Copilot 从 Demo 升级成「业务工作流的自动化操作员」。Week 11 起进入评测体系——让 Copilot 的「质量」从你拍脑袋的经验判断，变成可量化、可回归的工程指标。能办事，还得能证明它办得对。

WEEK 10 · THE END

你已经造好了一个能办事的 Copilot

从工具契约到 HITL 护栏，从行为血缘到端到端闭环——Week 1-10 第一次组装成一个能办事、受控、可审计的 Copilot

本周交付物（已 push 至 GitHub 仓库）

Tool Contracts

tools/*_contract.json

Tool Registry

tools/registry.py

Fallback Chain

tools/fallback.py

HITL Workflow

agent/hitl.py

Action Lineage

agent/lineage.py

E2E Demos

demos/e2e_*.py

下周预告

下周 → Week 11 评测体系 · RAGAS / DeepEval / Golden Set / 回归门禁 / LLM-as-Judge——让 Copilot 的质量从「经验判断」变成「可量化的工程指标」。能办事是这周的事，能证明它一直办得对，是下周的事。